Numyp Assignment

1. What is NumPy ? How to install it?

NumPy (Numerical Python) is a popular Python library used for numerical and scientific computing**.** It helps you work with arrays, matrices, and mathematical operations very efficiently.

Main uses:

- Fast array and matrix operations
- Mathematical functions (sin, cos, sqrt, etc.)
- Linear algebra
- Random number generation
- Base library for data science tools like Pandas, SciPy, and Matplotlib

To install NumPy:

Open Command Prompt / Terminal and run:

Pip install numpy

2. What is an array and how is it different from a list? What is the name of the built-in array class in NumPy ?

An array is a data structure that stores multiple values of the same data type in a single variable. The elements are stored in contiguous memory locations and are accessed using an index.In Python, arrays are commonly created using NumPy.

A list is a built-in Python data structure that can store multiple elements of different data types.

The name of the built-in array class in NumPy is ndarray.

ndarray stands for N-dimensional array**.**

3. What do you understand by rank of an ndarray?

The rank of an ndarray in NumPy means the number of dimensions (axes) the array has**.**

1-D Array (Rank = 1)

```
import numpy as np
a = np.array([1,2,3,4])
print(a.ndim)
```

2-D Array (Rank = 2)

```
b = np.array([[1,2,3],
              [4,5,6]])
print(b.ndim)
```

3-D Array (Rank = 3)

```
c = np.array([[[1,2],[3,4]],
              [[5,6],[7,8]]])
print(c.ndim)
```

4. Create the following NumPy arrays:
   a) A 1-D array called `zeros` having 10 elements and all the elements are set to zero.
   b) A 1-D array called `vowels` having the elements 'a', 'e', 'i', 'o' and 'u'.
   c) A 2-D array called ones having 2 rows and 5 columns and all the elements are set to 1 and dtype as int.
   d) Use nested Python lists to create a 2-D array called `myarray1` having 3 rows and 3 columns and store the following data:

2.7,	-2,	-19
0,	3.4,	99.9
10.6,	0,	13

   e) A 2-D array called `myarray2` using `arange()` having 3 rows and 5 columns with start value = 4, step size 4 and dtype as float.

```
import numpy as np

zeros=np.zeros(10)
print(zeros)
```

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

```
vowels=np.array(['a','e','i','o','u'])
print(vowels)
```

```
['a' 'e' 'i' 'o' 'u']
```

```
ones=np.ones((2,5),dtype=int)
print(ones)
```

```
[[1 1 1 1 1]
 [1 1 1 1 1]]
```

```
myarray1=np.array([[2.7,-2,-19],[0,3.4,99.9],[10.6,0,13]])
print(myarray1)
```

```
[[  2.7  -2.  -19. ]
 [  0.    3.4  99.9]
 [ 10.6   0.   13. ]]
```

```
myarray2=np.arange(4,4+15*4,4).reshape(3,5)
print(myarray2)
```

```
[[ 4  8 12 16 20]
 [24 28 32 36 40]
 [44 48 52 56 60]]
```

5. Using the arrays created in Question 4 above, write NumPy commands for the following:
   a) Find the dimensions, shape, size, data type of the items and itemsize of arrays `zeros`, `vowels`, `ones`, `myarray1` and `myarray2`.
   b) Reshape the array ones to have all the 10 elements in a single row.
   c) Display the 2nd and 3rd element of the array `vowels`.
   d) Display all elements in the 2nd and 3rd row of the array `myarray1`.
   e) Display the elements in the 1st and 2nd column of the array `myarray1`.
   f) Display the elements in the 1st column of the 2nd and 3rd row of the array `myarray1`.
   g) Reverse the array of `vowels`.

```
print(zeros.ndim)
print(zeros.shape)
print(zeros.size)
print(zeros.itemsize)
```

```
1
(10,)
10
8
```

```
print(ones.ndim)
print(ones.shape)
print(ones.size)
print(ones.itemsize)
```

```
2
(2, 5)
10
8
```

```
print(vowels.ndim)
print(vowels.shape)
print(vowels.size)
print(vowels.itemsize)
```

```
1
(5,)
5
4
```

```
print(myarray1.ndim)
print(myarray1.shape)
print(myarray1.size)
print(myarray1.itemsize)
```

```
2
(3, 3)
9
8
```

```
print(myarray2)
print(myarray2.ndim)
print(myarray2.shape)
print(myarray2.size)
print(myarray2.itemsize)
```

```
2
(3, 5)
15
8
```

```
# Reshape the array ones
arr=np.reshape(ones,(10,))
print(arr)
```

```
[1 1 1 1 1 1 1 1 1 1]
```

```
# Display elements
print(vowels[1:3])
print(myarray1[1:3])
print(myarray1[:,0:2])
print(myarray1[1:3,0])


# Reverse the array of vowels
print(vowels[::-1])
```

```
['e' 'i']
[[ 0.   3.4 99.9]
 [10.6  0.  13. ]]
[[ 2.7 -2. ]
 [ 0.   3.4]
 [10.6  0. ]]
[ 0.  10.6]
['u' 'o' 'i' 'e' 'a']
```

6. Using the arrays created in Question 4 above, write NumPy commands for the following:

a) Divide all elements of array ones by 3.

b) Add the arrays `myarray1` and `myarray2`.

c) Subtract `myarray1` from `myarray2` and store the result in a new array.

d) Multiply `myarray1` and `myarray2` elementwise.

e) Do the matrix multiplication of `myarray1` and `myarray2` and store the result in a new array `myarray3`.

f) Divide `myarray1` by `myarray2`.

g) Find the cube of all elements of `myarray1` and divide the resulting array by 2.

h) Find the square root of all elements of `myarray2` and divide the resulting array by 2. The result should be rounded to two places of decimals.

```
# Divide
ones_new=ones/3
print(ones_new)
```

```
[[0.33333333 0.33333333 0.33333333 0.33333333 0.33333333]
 [0.33333333 0.33333333 0.33333333 0.33333333 0.33333333]]
```

```
# shapes are different so we match column first
add=myarray1+myarray2[:,:3]
print(add)
```

```
[[  6.7   6.   -7. ]
 [ 24.   31.4 131.9]
 [ 54.6  48.   65. ]]
```

```
sub=myarray2[:,3]-myarray1
print(sub)
```

```
[[ 13.3   38.    75. ]
 [ 16.    32.6 -43.9]
 [  5.4   36.    43. ]]
```

```
mul=myarray1*myarray2[:,:3]
print(mul)
```

```
[[  10.8  -16.   -228. ]
 [   0.    95.2 3196.8]
 [ 466.4    0.    676. ]]
```

```
myarray3=myarray1@myarray2
print(myarray3)
```

```
[[ -873.2  -946.4 -1019.6 -1092.8 -1166. ]
 [ 4477.2  4890.4  5303.6  5716.8  6130. ]
 [  614.4   708.8   803.2   897.6   992. ]]
```

```
div=myarray1/myarray2[:,:3]
print(div)
```

```
[[ 0.675       -0.25        -1.58333333]
 [ 0.           0.12142857   3.121875  ]
 [ 0.24090909   0.           0.25      ]]
```

```
cube_myarray1=(myarray1**3)/2
print(cube_myarray1)
```

```
[[ 9.841500e+00 -4.000000e+00 -3.429500e+03]
 [ 0.000000e+00  1.965200e+01  4.985015e+05]
 [ 5.955080e+02  0.000000e+00  1.098500e+03]]
```

```
sqr_myarray2=np.round(np.sqrt(myarray2)/2,2)
print(sqr_myarray2)
```

```
[[1.   1.41 1.73 2.   2.24]
 [2.45 2.65 2.83 3.   3.16]
 [3.32 3.46 3.61 3.74 3.87]]
```

7. Using the arrays created in Question 4 above, write NumPy commands for the following:

   a) Find the transpose of `ones` and `myarray2`.
   b) Sort the array `vowels` in reverse.
   c) Sort the array `myarray1` such that it brings the lowest value of the column in the first row and so on.

```
# Transpose
print(ones.T)
```

```
[[1 1]
 [1 1]
 [1 1]
 [1 1]
 [1 1]]
```

```
#sort
print(np.sort(vowels)[::-1])
```

```
['u' 'o' 'i' 'e' 'a']
```

```
#sort
print(np.sort(myarray1,axis=0))# column wise
```

```
[[  0.   -2.  -19. ]
 [  2.7   0.   13. ]
 [ 10.6   3.4  99.9]]
```

8. Using the arrays created in Question 4 above, write NumPy commands for the following:

a) Use `NumPy. split()` to split the array `myarray2` into 5 arrays columnwise. Store your resulting arrays in `myarray2A`, `myarray2B`, `myarray2C`, `myarray2D` and `myarray2E`. Print the arrays `myarray2A`, `myarray2B`, `myarray2C`, `myarray2D` and `myarray2E`.

b) Split the array `zeros` at array index 2, 5, 7, 8 and store the resulting arrays in `zerosA`, `zerosB`, `zerosC` and `zerosD` and print them.

c) Concatenate the arrays `myarray2A`, `myarray2B` and `myarray2C` into an array having 3 rows and 3 columns.

```
# split
myarray2A,myarray2B,myarray2C,myarray2D,myarray2E=np.hsplit(myarray2,5)
print(myarray2A)
print(myarray2B)
print(myarray2C)
print(myarray2D)
print(myarray2E)
```

```
[[ 4]
 [24]
 [44]]
[[ 8]
 [28]
 [48]]
[[12]
 [32]
 [52]]
[[16]
 [36]
 [56]]
[[20]
 [40]
 [60]]
```

```
# Split before these indices → 2, 5, 7, 8
zerosA,zerosB,zerosC,zerosD,zerosE=np.split(zeros,[2,5,7,8])
print(zerosA)
print(zerosB)
print(zerosC)
print(zerosD)
print(zerosE)
```

```
[0. 0.]
[0. 0. 0.]
[0. 0.]
[0.]
[0. 0.]
```

```
con_arr=np.concatenate((myarray2A,myarray2B,myarray2C),axis=1)# column-wise
print(con_arr)
```

```
[[ 4  8 12]
 [24 28 32]
 [44 48 52]]
```

9. Create a 2-D array called `myarray4` using `arange()` having 14 rows and 3 columns with start value = -1, step size 0.25 having. Split this array row wise into 3 equal parts and print the result.

```
#create array
myarray4=np.arange(-1,-1+42*0.25,0.25).reshape(14,3)
print(myarray4)
```

```
[[-1.   -0.75 -0.5 ]
 [-0.25  0.    0.25]
 [ 0.5   0.75  1.  ]
 [ 1.25  1.5   1.75]
 [ 2.    2.25  2.5 ]
 [ 2.75  3.    3.25]
 [ 3.5   3.75  4.  ]
 [ 4.25  4.5   4.75]
 [ 5.    5.25  5.5 ]
 [ 5.75  6.    6.25]
 [ 6.5   6.75  7.  ]
 [ 7.25  7.5   7.75]
 [ 8.    8.25  8.5 ]
 [ 8.75  9.    9.25]]
```

```
#split
A,B,C=np.split(myarray4,[5,10])
print(A)
print(B)
print(C)
```

```
[[-1.   -0.75 -0.5 ]
 [-0.25  0.    0.25]
 [ 0.5   0.75  1.  ]
 [ 1.25  1.5   1.75]
 [ 2.    2.25  2.5 ]]
```

```
[[2.75 3.   3.25]
 [3.5  3.75 4.  ]
 [4.25 4.5  4.75]
 [5.   5.25 5.5 ]
 [5.75 6.   6.25]]
```

```
[[6.5  6.75 7.  ]
 [7.25 7.5  7.75]
 [8.   8.25 8.5 ]
 [8.75 9.   9.25]]
```

10. Using the `myarray4` created in the above questions, write commands for the following:

a) Find the sum of all elements.

b) Find the sum of all elements row wise.

---

c) Find the sum of all elements column wise.

d) Find the max of all elements.

e) Find the min of all elements in each row.

f) Find the mean of all elements in each row.

g) Find the standard deviation column wise.

```
# sum
print(myarray4.sum())
```

```
173.25
```

```
print(myarray4.sum(axis=1))#row-wise
```

```
[-2.25  0.    2.25  4.5   6.75  9.   11.25 13.5  15.75 18.   20.25 22.5
 24.75 27.  ]
```

```
print(myarray4.sum(axis=0))#column-wise
```

```
[54.25 57.75 61.25]
```

```
print(myarray4.max())
print(myarray4.min(axis=1))#row-sise
print(myarray4.mean(axis=1))#row-wise
print(myarray4.std(axis=0))#column-wise
```

```
9.25
[-1.   -0.25  0.5   1.25  2.    2.75  3.5   4.25  5.    5.75  6.    7.25
  8.    8.75]
[-0.75  0.    0.75  1.5   2.25  3.    3.75  4.5   5.25  6.    6.75  7.5
  8.25  9.  ]
[3.02334666 3.02334666 3.02334666]
```

Iris.txt Case Study Solved

The Iris dataset is a commonly used dataset in statistics and machine learning. It contains measurements of iris flowers from three different species: Setosa, Versicolor, and Virginica. Each sample contains four features: sepal length, sepal width, petal length, and petal width.

1. Load the data in the file `Iris.txt` in a 2-D array called `iris`.

```
import numpy as np
iris=np.genfromtxt("D:/Iris.txt",delimiter=",")
print(iris)
```

```
[[5.1 3.5 1.4 0.2 nan 1. ]
 [4.9 3.  1.4 0.2 nan 1. ]
 [4.7 3.2 1.3 0.2 nan 1. ]
 [4.6 3.1 1.5 0.2 nan 1. ]
 [5.  3.6 1.4 0.2 nan 1. ]
 [5.4 3.9 1.7 0.4 nan 1. ]
 [4.6 3.4 1.4 0.3 nan 1. ]
 [5.  3.4 1.5 0.2 nan 1. ]
 [4.4 2.9 1.4 0.2 nan 1. ]
 [4.9 3.1 1.5 0.1 nan 1. ]
 [5.5 2.6 4.4 1.2 nan 2. ]
 [6.1 3.  4.6 1.4 nan 2. ]
 [5.8 2.6 4.  1.2 nan 2. ]
 [5.  2.3 3.3 1.  nan 2. ]
 [5.6 2.7 4.2 1.3 nan 2. ]
 [5.7 3.  4.2 1.2 nan 2. ]
 [5.7 2.9 4.2 1.3 nan 2. ]
 [6.2 2.9 4.3 1.3 nan 2. ]
 [5.1 2.5 3.  1.1 nan 2. ]
 [5.7 2.8 4.1 1.3 nan 2. ]
 [6.9 3.1 5.4 2.1 nan 3. ]
 [6.7 3.1 5.6 2.4 nan 3. ]
 [6.9 3.1 5.1 2.3 nan 3. ]
 [5.8 2.7 5.1 1.9 nan 3. ]
 [6.8 3.2 5.9 2.3 nan 3. ]
 [6.7 3.3 5.7 2.5 nan 3. ]
 [6.7 3.  5.2 2.3 nan 3. ]
 [6.3 2.5 5.  1.9 nan 3. ]
 [6.5 3.  5.2 2.  nan 3. ]
 [6.2 3.4 5.4 2.3 nan 3. ]]
```

2. Drop column whose index = 4 from the array `iris`.

```
iris_new=np.delete(iris,4,axis=1)# for columns
print(iris_new)
```

```
[[5.1 3.5 1.4 0.2 1. ]
 [4.9 3.  1.4 0.2 1. ]
 [4.7 3.2 1.3 0.2 1. ]
 [4.6 3.1 1.5 0.2 1. ]
 [5.  3.6 1.4 0.2 1. ]
 [5.4 3.9 1.7 0.4 1. ]
 [4.6 3.4 1.4 0.3 1. ]
 [5.  3.4 1.5 0.2 1. ]
 [4.4 2.9 1.4 0.2 1. ]
 [4.9 3.1 1.5 0.1 1. ]
 [5.5 2.6 4.4 1.2 2. ]
 [6.1 3.  4.6 1.4 2. ]
 [5.8 2.6 4.  1.2 2. ]
 [5.  2.3 3.3 1.  2. ]
 [5.6 2.7 4.2 1.3 2. ]
 [5.7 3.  4.2 1.2 2. ]
 [5.7 2.9 4.2 1.3 2. ]
 [6.2 2.9 4.3 1.3 2. ]
 [5.1 2.5 3.  1.1 2. ]
 [5.7 2.8 4.1 1.3 2. ]
 [6.9 3.1 5.4 2.1 3. ]
 [6.7 3.1 5.6 2.4 3. ]
 [6.9 3.1 5.1 2.3 3. ]
 [5.8 2.7 5.1 1.9 3. ]
 [6.8 3.2 5.9 2.3 3. ]
 [6.7 3.3 5.7 2.5 3. ]
 [6.7 3.  5.2 2.3 3. ]
 [6.3 2.5 5.  1.9 3. ]
 [6.5 3.  5.2 2.  3. ]
 [6.2 3.4 5.4 2.3 3. ]]
```

3. Display the shape, dimensions and size of `iris`.

```
print(np.ndim(iris_new))
print(np.shape(iris_new))
print(np.size(iris_new))
```

```
2
(30, 5)
150
```

4. Split `iris` into three 2-D arrays, each array for a different species. Call them `iris1, iris2, iris3`.

```
iris1,iris2,iris3=np.split(iris_new,[10,20])
print("-------------------------------------------------------")
print(iris1)
print("-------------------------------------------------------")
print(iris2)
print("-------------------------------------------------------")
print(iris3)
print("-------------------------------------------------------")
```

5. Print the three arrays `iris1, iris2, iris3`

```
[[5.1 3.5 1.4 0.2 1. ]
 [4.9 3.  1.4 0.2 1. ]
 [4.7 3.2 1.3 0.2 1. ]
 [4.6 3.1 1.5 0.2 1. ]
 [5.  3.6 1.4 0.2 1. ]
 [5.4 3.9 1.7 0.4 1. ]
 [4.6 3.4 1.4 0.3 1. ]
 [5.  3.4 1.5 0.2 1. ]
 [4.4 2.9 1.4 0.2 1. ]
 [4.9 3.1 1.5 0.1 1. ]]
-------------------------------------------------------
[[5.5 2.6 4.4 1.2 2. ]
 [6.1 3.  4.6 1.4 2. ]
 [5.8 2.6 4.  1.2 2. ]
 [5.  2.3 3.3 1.  2. ]
 [5.6 2.7 4.2 1.3 2. ]
 [5.7 3.  4.2 1.2 2. ]
 [5.7 2.9 4.2 1.3 2. ]
 [6.2 2.9 4.3 1.3 2. ]
 [5.1 2.5 3.  1.1 2. ]
 [5.7 2.8 4.1 1.3 2. ]]
-------------------------------------------------------
[[6.9 3.1 5.4 2.1 3. ]
 [6.7 3.1 5.6 2.4 3. ]
 [6.9 3.1 5.1 2.3 3. ]
 [5.8 2.7 5.1 1.9 3. ]
 [6.8 3.2 5.9 2.3 3. ]
 [6.7 3.3 5.7 2.5 3. ]
 [6.7 3.  5.2 2.3 3. ]
 [6.3 2.5 5.  1.9 3. ]
 [6.5 3.  5.2 2.  3. ]
 [6.2 3.4 5.4 2.3 3. ]]
```

6. Create a 1-D array `header` having elements "sepal length", "sepal width", "petal length", "petal width", "Species No" in that order.

```
header=np.array(['sepal length','sepal width','petal length','petal width','species no'])
print(header)
```

7. Display the array `header`.

```
['sepal length' 'sepal width' 'petal length' 'petal width' 'species no']
```

8. Find the max, min, mean and standard deviation for the columns of the iris and store the results in the arrays `iris_max`, `iris_min`, `iris_avg`, `iris_std`, `iris_var` respectively. The results must be rounded to not more than two decimal places.

```
iris_max=np.round(iris_new.max(axis=0),2)# column wise
print(iris_max)
iris_min=np.round(iris_new.min(axis=0),2)# column wise
print(iris_min)
iris_avg=np.round(iris_new.mean(axis=0),2)# column wise
print(iris_avg)
iris_median=np.round(np.median(iris_new,axis=0),2)# column wise
print(iris_median)
iris_var=np.round(np.var(iris_new,axis=0),2) # column wise
print(iris_var)
iris_std=np.round(np.std(iris_new,axis=0),2)# column wise
print(iris_std)
```

```
[6.9 3.9 5.9 2.5 3. ]
[4.4 2.3 1.3 0.1 1. ]
[5.68 3.03 3.61 1.22 2.  ]
[5.7  3.   4.2  1.25 2.  ]
[0.58 0.12 2.74 0.67 0.67]
[0.76 0.35 1.65 0.82 0.82]
```

9. Similarly find the max, min, mean and standard deviation for the columns of the `iris1`, `iris2` and `iris3` and store the results in the arrays with appropriate names.

```
iris1_max=np.round(iris1.max(axis=0),2)# column wise
print(iris1_max)
iris1_min=np.round(iris1.min(axis=0),2)# column wise
print(iris1_min)
iris1_avg=np.round(iris1.mean(axis=0),2)# column wise
print(iris1_avg)
iris1_median=np.round(np.median(iris1,axis=0),2)# column wise
print(iris1_median)
iris1_var=np.round(np.var(iris1,axis=0),2) # column wise
print(iris1_var)
iris1_std=np.round(np.std(iris1,axis=0),2)# column wise
print(iris1_std)
```

```
[5.4 3.9 1.7 0.4 1. ]
[4.4 2.9 1.3 0.1 1. ]
[4.86 3.31 1.45 0.22 1.  ]
[4.9 3.3 1.4 0.2 1. ]
[0.08 0.08 0.01 0.01 0.  ]
[0.28 0.29 0.1  0.07 0.  ]
```

```
iris2_max=np.round(iris2.max(axis=0),2)# column wise
print(iris2_max)
iris2_min=np.round(iris2.min(axis=0),2)# column wise
print(iris2_min)
iris2_avg=np.round(iris2.mean(axis=0),2)# column wise
print(iris2_avg)
iris2_median=np.round(np.median(iris2,axis=0),2)# column wise
print(iris2_median)
iris2_var=np.round(np.var(iris2,axis=0),2) # column wise
print(iris2_var)
iris2_std=np.round(np.std(iris2,axis=0),2)# column wise
print(iris2_std)
```

```
[6.2 3.  4.6 1.4 2. ]
[5.  2.3 3.  1.  2. ]
[5.64 2.73 4.03 1.23 2.  ]
[5.7  2.75 4.2  1.25 2.  ]
[0.13 0.05 0.22 0.01 0.  ]
[0.36 0.22 0.47 0.11 0.  ]
```

```
iris3_max=np.round(iris3.max(axis=0),2)# column wise
print(iris3_max)
iris3_min=np.round(iris3.min(axis=0),2)# column wise
print(iris3_min)
iris3_avg=np.round(iris3.mean(axis=0),2)# column wise
print(iris3_avg)
iris3_median=np.round(np.median(iris3,axis=0),2)# column wise
print(iris3_median)
iris3_var=np.round(np.var(iris3,axis=0),2) # column wise
print(iris3_var)
iris3_std=np.round(np.std(iris3,axis=0),2)# column wise
print(iris3_std)
```

```
[6.9 3.4 5.9 2.5 3. ]
[5.8 2.5 5.  1.9 3. ]
[6.55 3.04 5.36 2.2  3.  ]
[6.7 3.1 5.3 2.3 3. ]
[0.11 0.06 0.08 0.04 0.  ]
[0.34 0.25 0.28 0.2  0.  ]
```

10. Check the minimum value for sepal length, sepal width, petal length and petal width of the three species in comparison to the minimum value of sepal length, sepal width, petal length and petal width for the data set as a whole and fill the table below with True if the species value is greater than the dataset value and False otherwise.

Feature	Iris Setosa	Iris Virginica	Iris Versicolor
Sepal Length	False	True	True
Sepal Width	True	True	False
Petal Length	False	True	True
Petal Width	False	True	True

```
ans1=iris_min[0]<iris1_min[0]
print(ans1)
ans1=iris_min[0]<iris3_min[0]
print(ans1)
ans1=iris_min[0]<iris2_min[0]
print(ans1)

print("----------------------------------------
ans1=iris_min[1]<iris1_min[1] # setosa
print(ans1)
ans1=iris_min[1]<iris3_min[1] # virginica
print(ans1)
ans1=iris_min[1]<iris2_min[1] # versicolor
print(ans1)

print("----------------------------------------
ans1=iris_min[2]<iris1_min[2] # setosa
print(ans1)
ans1=iris_min[2]<iris3_min[2] # virginica
print(ans1)
ans1=iris_min[2]<iris2_min[2] # versicolor
print(ans1)

print("----------------------------------------
ans1=iris_min[3]<iris1_min[3] # setosa
print(ans1)
ans1=iris_min[3]<iris3_min[3] # virginica
print(ans1)
ans1=iris_min[3]<iris2_min[3] # versicolor
print(ans1)
```

11. Compare Iris setosa's average sepal width to that of Iris virginica.

```
ans1=iris1_avg[1]==iris3_avg[1] # setosa & virginica
print(ans1)
```

False

12. Compare Iris setosa's average petal length to that of Iris virginica.

```
ans1=iris1_avg[2]==iris3_avg[2] # setosa & virginica
print(ans1)
```

False

```
ans1=iris1_avg[3]==iris3_avg[3] # setosa & virginica
print(ans1)
```

False

14. Save the array `iris_avg` in a comma separated file named `IrisMeanValues.txt` on the hard disk.

```
np.savetxt("D:/IrisMeanValues.txt",iris_avg,delimiter=",",fmt="%.2f")
```

```
5.68
3.03
3.61
1.22
2.00
```

15. Save the arrays `iris_max`, `iris_avg`, `iris_min` in a comma separated file named `IrisStat.txt` on the hard disk.

```
a=np.array([iris_max,iris_min,iris_avg])
np.savetxt("D:/IrisStat.txt",a,delimiter=",",fmt="%.2f")
```

```
6.90,3.90,5.90,2.50,3.00
4.40,2.30,1.30,0.10,1.00
5.68,3.03,3.61,1.22,2.00
```